



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 05

Problem Statement

Write a Python program to solve the $(n^2 - 1)$ -puzzle problem using the following search algorithms:

- **Uniform Cost Search (UCS)**
- **Bidirectional Search**

The program should:

1. Accept n as input ($n = 3$ for 8-puzzle, $n = 4$ for 15-puzzle).
2. Generate **10 random initial states** using a **Shuffle Method**:
 - Start with the Goal State.
 - Perform K random valid moves of the blank space to "shuffle" the board.
 - For $n = 3$, use $K = 20$. For $n = 4$, use $K = 20$.
 - Use the same 10 generated states for both algorithms to ensure a fair comparison.
3. Set a **strict 1-minute timeout** per algorithm per initial state.
4. Measure the execution time for each run. If a solution is not found within 60 seconds, record it as a **Timeout**.

Experimental Requirements

1. Constraints on Environment

To ensure the focus remains on algorithm logic, you **must not** use an IDE.

- Use a basic text editor (e.g., Notepad, Gedit, Vim) without code autocompletion or AI assistance.
- Execution and timing must be handled via the command line.

2. Data Collection & Plotting

For $n = 3$ and $n = 4$, generate a figure comparing the two algorithms:

- **Average Execution Time:** A graph showing the average time taken for successful runs. (Note: If an algorithm fails to solve a state within 1 minute, exclude that specific instance from the average calculation).
- **Search Efficiency:** Compare the number of nodes explored (nodes added to the "visited" set) by UCS vs. Bidirectional Search.

Key Technical Observations

Students should observe the following during their experiments:

- **UCS vs. BFS:** In Assignment 04, you implemented Breadth-First Search. Given that every move in the puzzle has a cost of 1, how does the behavior (and implementation) of UCS compare to BFS?
- **Bidirectional Intersection:** In Bidirectional Search, the search proceeds from both the Start and the Goal. How do you detect when the two searches meet? Does this meeting point always result in the shortest path?
- **The $n = 4$ Challenge:** Even with a "shuffled" state of only 20 moves, $n = 4$ presents a massive state space. Observe how the memory usage and time increase compared to $n = 3$.

Submission Requirements

- **Source Code:** A single Python file implementing the puzzle, the shuffle generator, and both search algorithms.
- **Report:** A PDF including:
 1. Graphs for Average Time and Nodes Explored for $n = 3$ and $n = 4$.
 2. A brief explanation of how you implemented the meeting condition for the Bidirectional Search.
 3. A discussion on the performance difference: Did Bidirectional Search solve the puzzles faster than UCS? Why or why not?